

Nama : Rival Mauli Prasetya

NPM : 252310009

Kelas : TI – 25 – PA2

TUGAS 13

Link Github : https://github.com/rivalmauli910-ctrl/252310009_LabAlgoritma.git

Pengertian Hashing

Hashing adalah teknik penyimpanan dan pencarian data dengan cara mengubah suatu nilai (key) menjadi alamat penyimpanan (index) menggunakan sebuah fungsi hash (hash function).

Tujuan hashing adalah agar proses pencarian, penambahan, dan penghapusan data dapat dilakukan dengan sangat cepat, bahkan rata-rata memiliki kompleksitas waktu $O(1)$.

Hashiing dibagi menjadi berapa?

1. Open Addressing

Pada metode ini seluruh data disimpan langsung di dalam tabel hash.

Jika terjadi collision, maka sistem akan mencari slot kosong lainnya menggunakan aturan tertentu.

2. Separate Chaining (Closed Addressing)

Pada metode ini setiap indeks tabel menyimpan sebuah linked list.

Jika terjadi collision, data baru dimasukkan ke linked list pada indeks tersebut.

Collision adalah keadaan ketika dua atau lebih data menghasilkan nilai hash yang sama, sehingga ingin menempati indeks yang sama pada tabel hash.

Penjelasan Kode

```
/*
```

```
 * Program Hashtable dengan OpenAddressing
```

```
 * Penampung data menggunakan array
```

```
 * Tiga Implementasi menggunakan Linear Probing, Quadric Probing, Double hashing
```

```
 * Populasi data menggunakan fungsi random
```

```
*/
```

```
#include <iostream> // Mengimpor pustaka untuk operasi input dan output standar (cin/cout)
```

```
#include <stdio.h> // Mengimpor pustaka I/O standar bahasa C
```

```
#include <math.h> // Mengimpor pustaka matematika, digunakan untuk fungsi sqrt()
```

```
#include <string>    // Mengimpor pustaka untuk menggunakan dan memanipulasi tipe data string
#include <cstdlib>    // Mengimpor pustaka utilitas umum, digunakan untuk fungsi rand() dan
srand()
#include <ctime>      // Mengimpor pustaka waktu, digunakan untuk memberikan 'seed' pada fungsi
random
```

```
using namespace std; // Menggunakan ruang nama standar (std) agar tidak perlu menulis std::
sebelum cin/cout
```

```
// asumsikan kita menggunakan array dengan batas 1024
```

```
int storage[1024]; // Mendeklarasikan array global bernama 'storage' berukuran 1024 sebagai
wadah hash table
```

```
int i = 0;         // Mendeklarasikan variabel global 'i' untuk menghitung jumlah percobaan (probing)
```

```
int hdt_boundary;  // Mendeklarasikan variabel global untuk menyimpan ukuran tabel (biasanya
bilangan prima)
```

```
void tambah_linear_probing(int n) // Fungsi untuk menyisipkan data 'n' menggunakan Linear Probing
```

```
{ // Awal blok fungsi
```

```
    bool inserted = false; // Variabel penanda apakah data sudah berhasil dimasukkan atau belum
```

```
    int hash; // Variabel untuk menyimpan nilai indeks/hash hasil perhitungan
```

```
    i = 0; // Mengatur ulang nilai percobaan (probing) menjadi 0
```

```
    while (!(inserted) && (i < hdt_boundary)) // Ulangi selama data belum masuk DAN percobaan
belum melebihi batas tabel
```

```
    { // Awal blok perulangan while
```

```
        hash = (n % hdt_boundary) + i; // Rumus Linear Probing: (nilai modulo ukuran tabel) ditambah
iterasi i
```

```
        if (storage[hash] == 0) // Mengecek apakah slot di indeks 'hash' masih kosong (bernilai 0)
```

```
        { // Awal blok if
```

```
            storage[hash] = n; // Jika kosong, masukkan nilai 'n' ke dalam slot tersebut
```

```
            inserted = true; // Ubah status inserted menjadi true karena data sudah masuk
```

```
        } // Akhir blok if
```

```

else          // Jika slot sudah terisi (terjadi tabrakan/collision)
{
    // Awal blok else
    ++i;      // Tambahkan nilai iterasi 'i' untuk mencoba indeks selanjutnya

    cout << "Terjadi tabrak di " << hash << endl; // Tampilkan pesan bahwa terjadi tabrakan di
    indeks tersebut

}            // Akhir blok else
}            // Akhir blok perulangan while

if (i == 0)   // Mengecek apakah data langsung masuk tanpa ada tabrakan (iterasi i tetap 0)
{
    // Awal blok if
    cout << "Langsung" << endl; // Tampilkan pesan "Langsung" jika tidak ada tabrakan
}
// Akhir blok if

cout << "Isi hash[" << hash << "]" dengan " << n << endl; // Tampilkan indeks tempat data 'n'
akhirnya disimpan

cout << "===== " << endl; // Garis
pemisah untuk kerapian output
}
// Akhir fungsi tambah_linear_probing

void tambah_quadric_probing(int n) // Fungsi untuk menyisipkan data 'n' menggunakan Quadratic
Probing
{
    // Awal blok fungsi
    bool inserted = false; // Variabel penanda apakah data sudah dimasukkan
    int hash;              // Variabel untuk menyimpan nilai indeks hash
    i = 0;                 // Mengatur ulang iterasi percobaan menjadi 0

    while (!(inserted) && (i < hdt_boundary)) // Ulangi selama belum masuk dan belum melebihi batas
    tabel
    {
        // Awal blok perulangan

        hash = (n % hdt_boundary) + (i * i); // Rumus Quadratic: (nilai modulo ukuran tabel) ditambah
        kuadrat dari iterasi i

        if (storage[hash] == 0) // Jika slot di indeks 'hash' kosong

```

```

{           // Awal blok if
    storage[hash] = n; // Simpan nilai 'n' di indeks tersebut
    inserted = true;  // Tandai bahwa data sudah berhasil masuk
}           // Akhir blok if
else        // Jika slot sudah terisi (tabrakan)
{           // Awal blok else
    ++i;      // Tambah nilai iterasi 'i'
    cout << "Terjadi tabrakan di " << hash << endl; // Informasikan letak tabrakan
}           // Akhir blok else
}           // Akhir blok perulangan

if (i == 0) // Jika langsung berhasil masuk tanpa tabrakan
{           // Awal blok if
    cout << "Langsung" << endl; // Cetak pesan "Langsung"
}           // Akhir blok if

cout << "Isi hash[" << hash << "]" dengan " << n << endl; // Cetak hasil akhir indeks penyimpanan
cout << "===== " << endl; // Garis
pemisah output
}           // Akhir fungsi tambah_quadric_probing

// fungsi pembantu double_hashing

int prima_atas(int n) // Fungsi untuk mencari bilangan prima terdekat yang lebih besar atau
sama dengan 'n'
{           // Awal fungsi prima_atas
    if (n % 2 == 0) // Cek apakah 'n' adalah bilangan genap
    {           // Awal blok if
        n++;      // Jika genap, jadikan ganjil dengan menambah 1 (karena prima selain 2 pasti
ganjil)
    }           // Akhir blok if
    else        // Jika 'n' sudah ganjil
    {           // Awal blok else

```

```

    n = n + 2;        // Tambah 2 untuk mengecek bilangan ganjil selanjutnya
}                    // Akhir blok else

bool ketemu = false; // Variabel penanda apakah bilangan prima sudah ditemukan
while (!ketemu)      // Lakukan perulangan selama bilangan prima belum ditemukan
{
    // Awal blok while pemburu prima
    bool prima = true; // Asumsikan bilangan saat ini (n) adalah prima
    if (n % 2 == 0)    // Cek lagi apakah genap (meskipun harusnya ganjil dari proses di atas)
    {
        // Awal blok if
        prima = false; // Jika genap, pasti bukan prima
    }
    // Akhir blok if
    else // Jika ganjil
    {
        // Awal blok else
        int i = 3;    // Mulai pembagi dari angka 3
        while (prima == true && i <= sqrt(n)) // Ulangi pembagian hingga akar kuadrat dari 'n'
        {
            // Awal blok while pengecekan prima
            if (n % i == 0) // Jika 'n' habis dibagi 'i'
            {
                // Awal blok if
                prima = false; // Maka 'n' bukan bilangan prima
            }
            // Akhir blok if

            i = i + 2; // Lanjut ke bilangan ganjil berikutnya sebagai pembagi
        }
        // Akhir blok while pengecekan prima
    }
    // Akhir blok else

    if (prima) // Jika status 'prima' masih true setelah dicek
    {
        // Awal blok if
        ketemu = true; // Hentikan perulangan pencarian prima (ketemu = true)
    }
    // Akhir blok if
    else // Jika ternyata bukan prima
    {
        // Awal blok else

```

```

        n = n + 2;    // Coba bilangan ganjil selanjutnya (n + 2)
    }                // Akhir blok else
}                    // Akhir blok while pemburu prima

return n;           // Kembalikan bilangan prima yang ditemukan sebagai hasil fungsi
}                    // Akhir fungsi prima_atas

int prima_bawah()    // Fungsi untuk mencari bilangan prima terdekat di bawah batas tabel
{                    // Awal fungsi prima_bawah
    int n = hdt_boundary; // Mulai pencarian dari ukuran batas tabel (hdt_boundary)
    if (n % 2 == 0)      // Jika ukurannya genap
    {                    // Awal blok if
        n--;            // Kurangi 1 agar menjadi ganjil
    }                    // Akhir blok if
    else                // Jika ukurannya sudah ganjil
    {                    // Awal blok else
        n = n - 2;      // Kurangi 2 untuk mengecek ganjil sebelumnya
    }                    // Akhir blok else

    bool ketemu = false; // Variabel penanda apakah sudah menemukan prima
    while (!ketemu)      // Ulangi selama prima belum ditemukan
    {                    // Awal blok while
        bool prima = true; // Asumsikan 'n' saat ini adalah prima
        if (n % 2 == 0)    // Jika genap (untuk safety check)
        {                  // Awal blok if
            prima = false; // Bukan prima
        }                  // Akhir blok if
        else                // Jika ganjil
        {                  // Awal blok else
            int i = 3;      // Mulai pembagi dari 3
            while (prima == true && i <= sqrt(n)) // Ulangi pengecekan pembagian sampai akar kuadrat 'n'

```

```

{          // Awal blok while cek prima
    if (n % i == 0) // Jika habis dibagi 'i'
    {          // Awal blok if
        prima = false; // Berarti bukan prima
    }          // Akhir blok if

    i = i + 2; // Coba pembagi ganjil berikutnya
}          // Akhir blok while cek prima
}          // Akhir blok else

if (prima)      // Jika lulus tes prima
{          // Awal blok if
    ketemu = true; // Tandai sudah ketemu
}          // Akhir blok if
else          // Jika tidak lulus tes
{          // Awal blok else
    n = n - 2; // Turun ke bilangan ganjil sebelumnya
}          // Akhir blok else
}          // Akhir blok while
return n;      // Kembalikan bilangan prima yang lebih kecil tersebut
}          // Akhir fungsi prima_bawah

```

void tambah_double_hashing(int n) // Fungsi untuk menyisipkan data 'n' dengan metode Double Hashing

```

{          // Awal fungsi

    cout << "===== " << endl; // Mencetak
    garis pemisah

    cout << "double hashing..." << endl; // Menginformasikan metode yang sedang digunakan

    int hash;          // Variabel penyimpan indeks tabel

    int bottom_prime = prima_bawah(); // Mengambil nilai prima di bawah ukuran tabel untuk fungsi
    hash kedua

    bool inserted = false; // Status keberhasilan penyisipan data

```

```

i = 0;           // Reset jumlah percobaan menjadi 0

while(!(inserted) && (i < hdt_boundary)) // Ulangi selama belum tersisip dan iterasi belum
melebihi tabel

{               // Awal blok while

    // Rumus Double Hashing: (Hash1(k) + i * Hash2(k)) mod ukuran_tabel

    hash = ((n % hdt_boundary) + i * ((n % bottom_prime + 1)) % hdt_boundary); // Menghitung
indeks dengan 2 fungsi hash

    if (storage[hash] == 0) // Jika slot kosong
    {
        // Awal blok if

        storage[hash] = n; // Masukkan data 'n' ke dalam slot

        inserted = true; // Tandai sudah dimasukkan
    }
    // Akhir blok if

    else // Jika slot penuh (tabrakan)
    {
        // Awal blok else

        ++i; // Tingkatkan jumlah percobaan

        cout << "Terjadi tabrakan di " << hash << endl; // Informasikan tabrakan
    }
    // Akhir blok else
}
// Akhir blok while


if (i == 0) // Jika masuk pada percobaan pertama
{
    // Awal blok if

    cout << "Langsung" << endl; // Cetak "Langsung"
}
// Akhir blok if


cout << "Isi hash[" << hash << "]" dengan " << n << endl; // Tampilkan indeks akhir data disisipkan

cout << "=====" << endl; // Garis
pemisah

} // Akhir fungsi


void cetak(int n) // Fungsi untuk mencetak isi tabel hash (menampilkan n data awal)
{
    // Awal fungsi cetak

```

```

cout << endl;          // Mencetak baris baru (enter)
cout << "Output program : " << endl; // Mencetak judul output
for (int a = 0; a < n; ++a) // Melakukan perulangan dari indeks 0 hingga n-1
{
    // Awal perulangan
    cout << "hash[" << a << "] = " << storage[a] << endl; // Mencetak isi array 'storage' di indeks 'a'
}
// Akhir perulangan
}
// Akhir fungsi cetak

int getRandomNumberFromRange(int min, int max) // Fungsi untuk menghasilkan angka acak antara
rentang min dan max
{
    // Awal fungsi
    return min + (rand() % (max - min)); // Rumus mendapatkan angka acak dalam rentang [min, max)
    menggunakan rand()
}
// Akhir fungsi

int main()          // Fungsi utama tempat program mulai dijalankan
{
    // Awal fungsi main
    int n, random_number; // Mendeklarasikan variabel jumlah data (n) dan penampung angka
    acak

    char pilihan;        // Mendeklarasikan variabel pilihan menu (karakter a, b, c)
    string cara;          // Mendeklarasikan variabel string penyimpan nama metode yang dipilih

    // harus panggil fungsi ini agar generate random datanya tidak sama terus.
    srand((unsigned) time(0)); // Memberi "seed" pada fungsi random berdasarkan waktu saat ini
    agar hasilnya selalu berbeda tiap kali program jalan

    cout << "Masukkan jumlah data : "; // Meminta user memasukkan jumlah data yang diinginkan
    cin >> n;          // Membaca masukan user dan menyimpannya di variabel 'n'

    cout << "Kami menggunakan metode open addressing ada 3 cara, " << // Menampilkan menu
    "pilih salah satu dengan memilih a, b, atau c : " << endl;    // Lanjutan teks menu
    cout << "a. linear probing" << endl; // Opsi a

```

```

cout << "b. quadric probing" << endl; // Opsi b
cout << "c. double hashing" << endl; // Opsi c
cout << "Pilih a, b, atau c ? " << endl; // Menanyakan pilihan user
cin >> pilihan;      // Membaca pilihan user (huruf a/b/c)

cout << endl << endl << "Proses pemasukkan data ke hashtable" << endl; // Memberikan baris
kosong dan info proses dimulai

// tentukan batas |T|

hdt_boundary = prima_atas(n); // Menentukan kapasitas tabel sebagai bilangan prima yang lebih
besar atau sama dengan n

for (int a = 0; a < n; ++a) // Memulai proses perulangan sebanyak 'n' kali untuk memasukkan data
{
    // Awal blok for
    random_number = getRandomNumberFromRange(0, n); // Hasilkan angka acak antara 0 hingga
'n'

    // switch mau pakai cara open addressing apa?
    switch(pilihan)    // Mengecek menu mana yang dipilih user
    {
        // Awal switch case
        case 'a':      // Jika user memilih 'a'
            tambah_linear_probing(random_number); // Panggil fungsi linear probing
            cara = "Linear Probing"; // Simpan nama metode di variabel 'cara'
            break;      // Keluar dari blok switch
        case 'b':      // Jika user memilih 'b'
            tambah_quadric_probing(random_number); // Panggil fungsi quadratic probing
            cara = "Quadric Probing"; // Simpan nama metode di variabel 'cara'
            break;      // Keluar dari blok switch
        case 'c':      // Jika user memilih 'c'
            tambah_double_hashing(random_number); // Panggil fungsi double hashing
            cara = "Double hashing"; // Simpan nama metode di variabel 'cara'
            break;      // Keluar dari blok switch
    }
    // Akhir switch case

```

```

}          // Akhir perulangan untuk memasukkan seluruh data acak

/*
 * Dibawah adalah operasi untuk cetak
 */

cetak(n);          // Memanggil fungsi cetak() untuk menampilkan indeks 0 sampai n-1

cout << "Diatas adalah hasil output Hashtable dengan cara \" << cara << "\" << endl; //
Menampilkan pesan penutup dengan nama metode

switch(pilihan)    // Mengecek kembali pilihan user untuk menampilkan rumus matematikanya
{
    // Awal switch case

    case 'a':       // Jika linear probing

        cout << "h(k) = (k mod " << hdt_boundary << ") + i" << endl; // Cetak rumus linear probing

        break;      // Keluar dari switch

    case 'b':       // Jika quadratic probing

        cout << "h(k) = (k mod " << hdt_boundary << ") + i * i" << endl; // Cetak rumus quadratic
        probing

        break;      // Di kode asal tidak ada break di sini, tapi di case 'c' ada, sehingga jika milih 'b',
        ia akan lanjut cetak case 'c' juga (Fallthrough behavior)

    case 'c':       // Jika double hashing

        cout << "h1(k) = k mod " << hdt_boundary << endl; // Cetak rumus fungsi hash pertama

        cout << "h2(k) = (h1(k) + i * (k mod " << prima_bawah() << " + 1) ) mod " << hdt_boundary <<
        endl; // Cetak rumus fungsi hash kedua

        break;      // Keluar dari switch

    }              // Akhir switch case
}                // Akhir dari program (fungsi main)

```